

# Практическая теория типов

## Лекция 9: Системы типов высших порядков: $\lambda\omega$ , $\lambda P$

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

# План лекции

- 1 Система  $\lambda\omega$
- 2 Зависимые типы и система  $\lambda P$
- 3 Зависимые суммы, уточнения и индуктивные типы

- 1 Система  $\lambda\omega$
- 2 Зависимые типы и система  $\lambda P$
- 3 Зависимые суммы, уточнения и индуктивные типы

# Мотивация: ограничения системы $\lambda_{\rightarrow}$

## Ограничения выразительности

В просто типизированном  $\lambda$ -исчислении  $\lambda_{\rightarrow}$  отсутствуют:

- Абстракция на уровне типов: невозможно определить  $\lambda\alpha. \tau$
- Параметрический полиморфизм: типы не могут зависеть от типовых параметров
- Конструкторы типов: отсутствуют отображения вида  $\kappa \rightarrow \kappa'$

## Практические применения

- Параметрические типы данных: универсальный конструктор списков  
 $\text{List} : \star \rightarrow \star$ , где  $\text{List Int}$ ,  $\text{List String}$ ,  $\text{List (List Bool)}$
- Функторы: полиморфные операции над параметризованными типами  
 $\text{map} : \forall\alpha, \beta. (\alpha \rightarrow \beta) \rightarrow \text{List } \alpha \rightarrow \text{List } \beta$
- Монадические конструкторы:  $\text{Maybe}$ ,  $\text{Either}$ ,  $\text{IO}$  как конструкторы вида  $\star \rightarrow \star$

# Решение: расширение до $\lambda\omega$

## Расширение системы типов

Система  $\lambda\omega$  расширяет  $\lambda_{\rightarrow}$  введением зависимости типов от типов, что формализуется через конструкторы типов (type constructors) и систему видов (kinds).

Система  $\lambda_{\rightarrow}$ :

- Абстракция: термы по термам
- $\lambda x : \text{Int}. x + 1$
- Типы фиксированы:  $\text{Int}$ ,  $\text{Bool}$ ,  $\text{Int} \rightarrow \text{Int}$

Система  $\lambda\omega$ :

- Абстракция: типы по типам
- $\lambda\alpha : \star. \alpha \rightarrow \alpha$
- Виды (kinds):  $\star, \star \rightarrow \star$

## Корректность по видам

Не всякое применение конструкторов типов корректно. Система сортов  $\{\star, \square\}$  и видов обеспечивает статическую верификацию корректности конструкторов (well-kindedness).

## Два расширения $\lambda_{\rightarrow}$

- $\lambda\omega$ : типы зависят от типов (конструкторы типов, виды)
- System F ( $\lambda 2$ ): параметрический полиморфизм ( $\forall$ ,  $\Lambda$ )
- System  $F_\omega$ : объединение  $\lambda\omega + \lambda 2$

### Чистая $\lambda\omega$ :

- Типовые абстракции:  $\lambda\alpha : \star. \tau$
- Виды:  $\kappa ::= \star \mid \kappa \rightarrow \kappa$
- Нет  $\forall$

### System $F_\omega$ :

- Всё из  $\lambda\omega$  плюс
- Полиморфизм:  $\forall\alpha^\kappa. \tau$
- Термовые абстракции:  $\Lambda\alpha^\kappa. M$

## В этой лекции

Основной фокус — на  $\lambda\omega$  (виды и конструкторы). Примеры с  $\forall$  относятся к  $F_\omega$ .

# Сорты: формальное определение

## Определение

Сорты (sorts) — элементы двухэлементного множества  $\mathcal{S} = \{\star, \square\}$ , классифицирующие выражения уровня типов:

- $\star$  — сорт собственных типов (proper types):  $\Gamma \vdash \tau : \star$  означает, что  $\tau$  — тип, обитаемый термами
- $\square$  — сорт видов (kinds):  $\Gamma \vdash \kappa : \square$  означает, что  $\kappa$  — вид (тип конструкторов)
- Конструктор типов —  $\lambda$ -абстракция на уровне типов:  $\lambda\alpha : \kappa. \tau$
- Конструктор правильный (proper), если его вид  $\kappa_1 \rightarrow \dots \rightarrow \kappa_n \rightarrow \star$
- Система сортов обеспечивает well-kindedness: корректность применения конструкторов

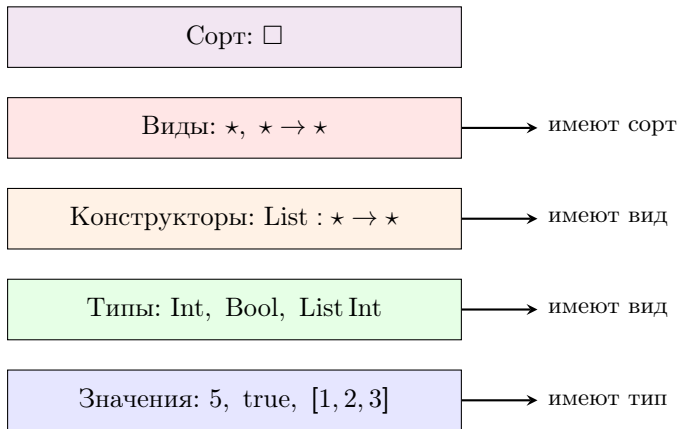
# Иерархия уровней абстракции

| L1                                                                   | L2                                         | L3                                | L4        |
|----------------------------------------------------------------------|--------------------------------------------|-----------------------------------|-----------|
| термы                                                                | типы/конструкторы                          | виды                              | сорты     |
| $\lambda x : \alpha. x$<br>term                                      | $\alpha \rightarrow \alpha$<br>proper type | $\star$<br>kind                   | $\square$ |
| $\lambda \beta : \star. \beta \rightarrow \beta$<br>type constructor | —                                          | $\star \rightarrow \star$<br>kind | $\square$ |

- Уровень 1: термы — вычисляемые выражения программ
- Уровень 2: собственные типы и конструкторы типов
- Уровень 3: виды — классификаторы конструкторов типов
- Уровень 4: сорта — метаяровень классификации видов



# Интуиция: лестница абстракций



- $\lambda\omega$  позволяет абстракцию на уровне L2:  $\lambda\alpha : \star. \dots$
- Система видов (L3) проверяет корректность конструкторов

# Правила вывода системы $\lambda\omega$

$$\begin{array}{c} \frac{}{\vdash \star : \square} \text{ (axiom)} \quad \frac{\Gamma \vdash A : s \quad x \notin \Gamma}{\Gamma, x:A \vdash x : A} \text{ (var)} \quad \frac{\Gamma \vdash M : B \quad \Gamma \vdash A : s \quad x \notin \Gamma}{\Gamma, x:A \vdash M : B} \text{ (weak)} \\[10pt] \frac{\Gamma \vdash A : s \quad \Gamma \vdash B : s}{\Gamma \vdash A \rightarrow B : s} \text{ (}\rightarrow\text{-form)} \quad \frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash A \rightarrow B : s}{\Gamma \vdash \lambda x:A. M : A \rightarrow B} \text{ (}\lambda\text{-intro)} \\[10pt] \frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B} \text{ (app)} \quad \frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s \quad A =_{\beta} B}{\Gamma \vdash M : B} \text{ (conv}_{\beta}\text{)} \end{array}$$

Замечание: параметр  $s \in \{\star, \square\}$  позволяет правилам работать как на уровне типов, так и на уровне видов. Типовые абстракции  $\lambda\alpha : \kappa. \tau$  и применения  $F A$  выводятся теми же правилами.

# Правила для видов (эскиз $\lambda\omega$ )

$$\begin{array}{c} \frac{}{\vdash \star : \square} \text{ (axiom)} \qquad \frac{\Gamma \vdash \kappa_1 : \square \quad \Gamma \vdash \kappa_2 : \square}{\Gamma \vdash \kappa_1 \rightarrow \kappa_2 : \square} \text{ (kind-form)} \qquad \frac{\alpha : \kappa \in \Gamma}{\Gamma \vdash \alpha : \kappa} \text{ (kvar)} \\[2ex] \frac{\Gamma, \alpha : \kappa_1 \vdash \tau : \kappa_2}{\Gamma \vdash \lambda\alpha : \kappa_1. \tau : \kappa_1 \rightarrow \kappa_2} \text{ (kind-}\lambda\text{-intro)} \qquad \frac{\Gamma \vdash F : \kappa_1 \rightarrow \kappa_2 \quad \Gamma \vdash A : \kappa_1}{\Gamma \vdash F A : \kappa_2} \text{ (kind-app)} \end{array}$$

Эти правила делают явной типовую  $\lambda$ -абстракцию и аппликацию (конструкторы типов).

## Теорема (Консервативность)

Система  $\lambda\omega$  — консервативное расширение  $\lambda_{\rightarrow}$ : для всех  $\Gamma, M, \tau$ , если  $\Gamma \vdash_{\lambda_{\rightarrow}} M : \tau$ , то  $\Gamma \vdash_{\lambda\omega} M : \tau$ .

## Теорема (Субъект-редукция)

Если  $\Gamma \vdash M : A$  и  $M \rightarrow_{\beta} N$ , то  $\Gamma \vdash N : A$ .

## Теорема (Сильная нормализация (Girard))

Все типизированные термы  $\lambda\omega$  завершаются: нет бесконечных  $\beta$ -редукций.

# Выразительность типов в $\lambda\omega$

## Арифметика типов

Алгебраические типы данных естественно выражаются через полиномиальные функторы:

- Константы: 0 (пустой тип), 1 (единичный тип)
- Операции:  $+$  (сумма),  $\times$  (произведение),  $\rightarrow$  (экспонента)

## Что добавляет $\lambda\omega$

Типовая абстракция и композиция конструкторов:

- $\lambda\alpha : \kappa. \tau$  — типовые функции
- Композиция:  $\text{Compose } F \ G \ A = F \ (G \ A)$
- НКТ: конструкторы как параметры  $(\star \rightarrow \star) \rightarrow \star \rightarrow \star$

Замечание: условные операторы на уровне типов требуют зависимых типов (type families, DataKinds в GHC).

# Примеры конструкторов типов (в $F_\omega$ )

## Параметрические типы данных

- Конструктор списков:  $\text{List} : \star \rightarrow \star$   
 $\text{List} \equiv \lambda\alpha : \star. \forall\beta. (\alpha \rightarrow \beta \rightarrow \beta) \rightarrow \beta \rightarrow \beta$
  - Конструктор пар:  $\text{Pair} : \star \rightarrow \star \rightarrow \star$   
 $\text{Pair} \equiv \lambda\alpha : \star. \lambda\beta : \star. \forall\gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$
  - Функторы высшего порядка:  $(\star \rightarrow \star) \rightarrow (\star \rightarrow \star)$
- 
- Конструкторы типов реализуют абстракцию на уровне типов
  - Виды обеспечивают статическую корректность применения конструкторов
  - Эти примеры используют  $\forall$ , поэтому относятся к  $F_\omega$

## Иерархия абстракции

Система  $\lambda\omega$  вводит многоуровневую иерархию абстракции:

- 1 Уровень 0 — константы: `Int`, `Bool`, `String`  
Вид:  $\star$  (собственные типы)
- 2 Уровень 1 — простые конструкторы: `List`, `Maybe`, `Tree`  
Вид:  $\star \rightarrow \star$  (принимают тип, возвращают тип)  
Пример: `List Int :  $\star$`
- 3 Уровень 2 — конструкторы высших порядков: принимают конструкторы  
Вид:  $(\star \rightarrow \star) \rightarrow \dots$   
Пример: композиция, функторы, монады

# Проблема: абстракция над конструкторами

## Мотивирующий пример

Рассмотрим задачу написания обобщённой функции `map`.

Для списков:

$$\text{mapList} : \forall \alpha, \beta. (\alpha \rightarrow \beta) \rightarrow \text{List } \alpha \rightarrow \text{List } \beta$$

Для деревьев:

$$\text{mapTree} : \forall \alpha, \beta. (\alpha \rightarrow \beta) \rightarrow \text{Tree } \alpha \rightarrow \text{Tree } \beta$$

## Вопрос

Возможна ли абстракция над `List` и `Tree` с написанием единой функции `map` для любого конструктора вида  $\star \rightarrow \star$ ?

Ответ: такая абстракция возможна, если язык поддерживает типы высших порядков (Higher-Kinded Types).



# Полиморфизм vs НКТ

Обычный полиморфизм (System F):

Абстракция над типами:

$$\forall \alpha. \alpha \rightarrow \alpha$$

НКТ (System  $F_\omega$ ):

Абстракция над конструкторами:

$$\forall F : \star \rightarrow \star. \dots$$

## Пример (Java/C#)

```
<T> T identity(T x) {  
    return x;  
}  
// T is a type variable
```

$\alpha$  имеет вид  $\star$

## Пример (Haskell)

```
class Functor f where  
    fmap :: (a -> b)  
         -> f a -> f b  
-- f is a constructor variable
```

$F$  имеет вид  $\star \rightarrow \star$

# Почему не все языки поддерживают НКТ?

## Технические сложности

### ❶ Усложнение системы типов:

- Требуется полноценная система видов (kind system)
- Вывод типов становится сложнее — нужен вывод видов
- Проверка типов требует проверки корректности по видам

### ❷ Реализация в компиляторе:

- Нужна поддержка на всех этапах: парсинг, type checking, кодогенерация
- Type erasure усложняется — виды должны стираться корректно

### ❸ Обратная совместимость:

- Сложно добавить НКТ в существующий язык без breaking changes
- Java: дженерики с erasure и формат байткода JVM усложняют нативные НКТ, потребовались бы изменения спецификаций

# Языки с поддержкой НКТ

## Полная поддержка:

- Haskell: native НКТ
- Scala 3: native НКТ
- PureScript: native НКТ
- OCaml: модули высших порядков (иной механизм)

## Эмуляция/ограничения:

- Scala 2: type lambdas (kind projector)
- TypeScript: паттерны, частичная эмуляция
- Rust: GATs, associated types (частично)
- Java/C#: нет поддержки

## Пример эмуляции в Scala 2

```
// Emulation via type lambda (kind-projector plugin)
type ~>[F[_], G[_]] = ...
```

Вывод: НКТ требуют серьёзной поддержки на уровне системы типов языка.

# Типы высших порядков (Higher-Kinded Types)

## Определение

Higher-Kinded Types (НКТ) — конструкторы вида  $(\kappa_1 \rightarrow \kappa_2) \rightarrow \kappa_3$ , где  $\kappa_i$  — виды.  
Формально: конструкторы, параметризованные конструкторами.

Обычные типы ( $\star$ ):

- $\text{Int} : \star$
- $\text{Bool} : \star$
- $\text{String} : \star$

Типы вида  $\star$ : выражения, обитаемые термами.

Конструкторы типов ( $\star \rightarrow \star$ ):

- $\text{List} : \star \rightarrow \star$
- $\text{Maybe} : \star \rightarrow \star$
- $\text{Tree} : \star \rightarrow \star$

Конструкторы вида  $\star \rightarrow \star$ : отображения из типов в типы.

## Принцип абстракции

НКТ обеспечивают полиморфизм высших порядков: абстракцию над конструкторами, а не только над типами.

## Примеры видов (kinds)

| Конструктор | Вид                                         | Применение                  |
|-------------|---------------------------------------------|-----------------------------|
| Int         | $\star$                                     | —                           |
| List        | $\star \rightarrow \star$                   | List Int : $\star$          |
| Pair        | $\star \rightarrow \star \rightarrow \star$ | Pair Int Bool : $\star$     |
| Either      | $\star \rightarrow \star \rightarrow \star$ | Either String Int : $\star$ |

### Правило применения

Если  $\Gamma \vdash F : \kappa_1 \rightarrow \kappa_2$  и  $\Gamma \vdash A : \kappa_1$ , то  $\Gamma \vdash F A : \kappa_2$ .

# НКТ: конструктор Const

## Определение

Конструктор Const игнорирует второй аргумент:

$$\text{Const} \equiv \lambda\alpha : \star. \lambda\beta : \star. \alpha \qquad \text{Const} : \star \rightarrow \star \rightarrow \star$$

Примеры:

- $\text{Const Int Bool} \equiv \text{Int}$
- $\text{Const Int} : \star \rightarrow \star$  — частично применённый

## Применение в Haskell

```
newtype Const a b = Const { getConst :: a }  
-- Const Int Bool ~ Int
```

## Определение

Конструктор `Compose` композитует два конструктора типов:

$$\text{Compose} \equiv \lambda F : \star \rightarrow \star. \lambda G : \star \rightarrow \star. \lambda A : \star. F (G A)$$

Вид:  $(\star \rightarrow \star) \rightarrow (\star \rightarrow \star) \rightarrow \star \rightarrow \star$

Примеры:

- $\text{Compose List Maybe Int} \equiv \text{List (Maybe Int)}$
- $\text{Compose Maybe List String} \equiv \text{Maybe (List String)}$

## Применение в Haskell

```
newtype Compose f g a = Compose { getCompose :: f (g a) }  
-- Compose [] Maybe Int ~ [Maybe Int]
```

## Пример вывода вида (в $F_\omega$ ): Pair

### Задача

Вывести вид для  $\text{Pair} \equiv \lambda\alpha : \star. \lambda\beta : \star. \forall\gamma : \star. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$

- 1 По правилу (var):  $\alpha : \star, \beta : \star \vdash \alpha : \star$  и  $\dots \vdash \beta : \star$
- 2 По правилу ( $\rightarrow$ -form):  $\alpha : \star, \beta : \star \vdash \alpha \rightarrow \beta \rightarrow \gamma : \star$
- 3 Аналогично для остальной части типа:  $\dots \vdash \forall\gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma : \star$
- 4 По правилу ( $\lambda$ -intro):  $\alpha : \star \vdash \lambda\beta : \star. \dots : \star \rightarrow \star$
- 5 Снова ( $\lambda$ -intro):  $\vdash \lambda\alpha : \star. \lambda\beta : \star. \dots : \star \rightarrow \star \rightarrow \star$

Результат:  $\text{Pair} : \star \rightarrow \star \rightarrow \star$



# Контрпримеры: некорректные применения конструкторов

## Проблема: несоответствие видов

Система видов отклоняет применения конструкторов, нарушающие корректность по видам (well-kindedness).

## Контрпример 1: применение типа к типу

Пусть  $\text{Int} : \star$  и  $\text{Bool} : \star$ .

Попытка:  $\text{Int Bool}$

Ошибка:  $\text{Int}$  имеет вид  $\star$ , но применение требует вида  $\kappa \rightarrow \kappa'$ .

$$\frac{\Gamma \vdash \text{Int} : \star \quad \Gamma \vdash \text{Bool} : \star}{\Gamma \vdash \text{Int Bool} : ???} \quad (\rightarrow\text{-elim})$$

**Отклонено:** вид  $\star$  не является функциональным.

# Контрпримеры: несоответствие аргументности

## Контрпример 2: недостаточно аргументов

Пусть  $\text{Pair} : \star \rightarrow \star \rightarrow \star$ .

Попытка использовать  $\text{Pair Int}$  как собственный тип:

$$\lambda x : \text{Pair Int}. x$$

Ошибка:  $\text{Pair Int} : \star \rightarrow \star$ , но аннотация типа требует  $\star$ .

**Отклонено:** конструктор не полностью применён (частичное применение).

## Контрпример 3: несоответствие порядка

Пусть  $\text{List} : \star \rightarrow \star$  и  $\text{Maybe} : \star \rightarrow \star$ .

Попытка:  $\text{List Maybe}$

Ошибка:  $\text{List}$  ожидает аргумент вида  $\star$ , но  $\text{Maybe} : \star \rightarrow \star$ .

**Отклонено:** несоответствие видов ( $\star \neq \star \rightarrow \star$ ).

# Вывод вида для Compose

Докажем, что  $\text{Compose} : (\star \rightarrow \star) \rightarrow (\star \rightarrow \star) \rightarrow \star \rightarrow \star$ .

$$\frac{\frac{\frac{F : \star \rightarrow \star \in \Gamma}{\Gamma \vdash F : \star \rightarrow \star} (\text{var}) \quad \frac{G : \star \rightarrow \star \in \Gamma}{\Gamma \vdash G : \star \rightarrow \star} (\text{var}) \quad \frac{A : \star \in \Gamma}{\Gamma \vdash A : \star} (\text{var})}{\Gamma \vdash G A : \star} (\rightarrow\text{-elim})}{\Gamma \vdash F (G A) : \star} (\rightarrow\text{-elim})}{\Gamma, F, G \vdash \lambda A : \star. F (G A) : \star \rightarrow \star} (\rightarrow\text{-intro})}{\Gamma, F \vdash \lambda G : \star \rightarrow \star. \lambda A : \star. F (G A) : (\star \rightarrow \star) \rightarrow \star \rightarrow \star} (\rightarrow\text{-intro})}{\Gamma \vdash \lambda F : \star \rightarrow \star. \lambda G : \star \rightarrow \star. \lambda A : \star. F (G A) : (\star \rightarrow \star) \rightarrow (\star \rightarrow \star) \rightarrow \star \rightarrow \star} (\rightarrow\text{-intro})$$

# НКТ и функторы (в $F_\omega$ )

## Класс типов Functor

Функтор — конструктор  $F : \star \rightarrow \star$  с операцией `fmap`:

$$\text{fmap} : \forall \alpha, \beta. (\alpha \rightarrow \beta) \rightarrow F \alpha \rightarrow F \beta$$

Примеры: `List`, `Maybe`, `Tree`

## Haskell

```
class Functor (f :: Type -> Type) where
  fmap :: (a -> b) -> f a -> f b
-- Requires НКТ: f has kind Type -> Type
```

Без НКТ невозможно выразить абстракцию над функторами.

## Теорема

Если  $F$  и  $G$  — функторы, то  $\text{Compose } F \ G$  — тоже функтор.

## Реализация в Haskell

```
instance (Functor f, Functor g) => Functor (Compose f g) where
  fmap h (Compose fga) = Compose (fmap (fmap h) fga)
-- Example: fmap (+1) (Compose [Just 1, Nothing])
--           = Compose [Just 2, Nothing]
```

Интуиция: `fmap` применяется дважды — для  $F$  и для  $G$ .

# Виды в Haskell: история и реализация

## Встроенная система видов

В Haskell виды (kinds) — встроенная часть системы типов, используемая компилятором для проверки корректности конструкторов.

Эволюция:

- Haskell 98: базовая система  $\star$  и  $\star \rightarrow \star$
- GHC 6.x: KindSignatures — явные аннотации
- GHC 8.0+: PolyKinds, DataKinds
- Современный GHC: Type из Data.Kind

## Явные аннотации видов

```
{-# LANGUAGE KindSignatures #-}  
data Proxy (a :: Type) = Proxy  
data App (f :: Type -> Type) (a :: Type) = App (f a)
```

# Виды в компиляторе GHC

Роль видов при компиляции:

- ❶ Проверка корректности (kind checking):
  - Компилятор выводит виды для всех конструкторов
  - Отклоняет ошибки: `Maybe Int Int`
- ❷ Вывод видов (kind inference):
  - Автоматический вывод (Hindley-Milner на уровне видов)
- ❸ Type erasure:
  - Виды стираются после проверки

## Пример вывода видов

```
ghci> :kind Maybe
Maybe :: Type -> Type
-- Note: * is a deprecated synonym for Type
```

# Современные расширения: DataKinds

## DataKinds: продвижение типов в виды

Расширение DataKinds позволяет использовать типы данных как виды.

## Пример: типобезопасные векторы

```
{-# LANGUAGE DataKinds, GADTs #-}  
import Data.Kind (Type)  
data Nat = Zero | Succ Nat  
-- Vec has kind: Type -> Nat -> Type  
data Vec (a :: Type) (n :: Nat) where  
  Nil  :: Vec a 'Zero  
  Cons :: a -> Vec a n -> Vec a ('Succ n)  
safeHead :: Vec a ('Succ n) -> a  
safeHead (Cons x _) = x
```

Виды гарантируют корректность на этапе компиляции.



## Некорректное применение конструктора

```
type Wrong = Maybe Int Bool -- Error!  
-- Maybe :: Type -> Type  
-- Int :: Type  
-- Bool :: Type  
-- Maybe Int :: Type (OK)  
-- (Maybe Int) Bool :: kind error!
```

## Корректное применение

```
type Correct1 = Maybe Int      -- OK: Type  
type Correct2 = Either Int Bool -- OK: Type  
type Correct3 = [Maybe Int]   -- OK: Type
```

## Теоретическая основа

Система видов GHC реализует фрагмент  $\lambda\omega$ :

- Виды Haskell изоморфны типам конструкторов в  $\lambda\omega$
- Kind checking реализует правила  $\lambda\omega$ : axiom,  $\rightarrow$ -form,  $\lambda$ -intro, app

$\lambda\omega$  (теория):

- $\star : \square$
- $\lambda\alpha : \star. \alpha \rightarrow \alpha : \star \rightarrow \star$
- Формальные правила вывода

Haskell (практика):

- В GHC: `Type :: TYPE 'LiftedRep`, `*`  
— синоним `Type`
- `type Id a = a`
- Алгоритм вывода видов в GHC

## Практическая значимость

Формальное изучение  $\lambda\omega$  обеспечивает строгое понимание системы типов Haskell и корректности kind checking в GHC.

# План лекции

- 1 Система  $\lambda\omega$
- 2 Зависимые типы и система  $\lambda P$
- 3 Зависимые суммы, уточнения и индуктивные типы

## Зависимые типы: другой «угол» куба

### Контекст

Дальше мы переходим от  $F_\omega$  (типовые конструкторы и полиморфизм по типам) к системам с зависимостью типов от термов ( $\Pi$ ,  $\Sigma$ , индуктивные типы). Это другое измерение «куба Ламбы».

### Что изменяется

- В  $\lambda\omega$  и  $F_\omega$ : типы зависят от типов
- В  $\lambda P$  и далее: типы зависят от термов
- Примеры:  $\text{Vec}(n)$ ,  $\Pi x : A. B(x)$

- Тип массива фиксированного размера.  
Естественно выразим зависимым типом:

$$\lambda n : \mathbb{N}. \text{Vec}(n)$$

- Propositions-as-Types (PAT).  
 $\lambda n : \mathbb{N}. P(n)$ , где  $P$  — утверждение (в логике — предикат)
- Результат зависит от аргумента.  
Некоторые математические идеи выражаемы только через зависимые типы: когда тип результата функции зависит от её аргумента.

## Мотивация: универсальные утверждения

- Как формально доказать: если число делится на 4, то оно чётное.
- Переформулируем как тип:  $\text{isDivFour}(n) \rightarrow \text{isEven}(n)$ .
- Достаточно построить терм  $t$ , такой что

$$t(n) : \text{isDivFour}(n) \rightarrow \text{isEven}(n).$$

- Но нам нужно выразить, что это верно для любого  $n$ :

$$t : \prod n : \mathbb{N}. \text{isDivFour}(n) \rightarrow \text{isEven}(n).$$

## Пи-тип (Π-тип): определение

- Π-тип (dependent product / dependent function type):

$$\prod_{x:A} B(x) \quad \equiv \quad \Pi_{x:A}. B(x)$$

- Это тип функций, которые каждому  $x : A$  сопоставляют элемент типа  $B(x)$ .
- Пусть существует вселенная  $U$ ,  $A : U$ ,  $B(x) : U$ . Тогда можно говорить о семействе типов  $B : A \rightarrow U$ .
- Если  $B : A \rightarrow U$  — константная функция (т.е.  $B$  не зависит от  $x$ ), то

$$\Pi_{x:A}. B \simeq A \rightarrow B.$$

- Как декартово произведение семейств.  
Если  $A = \text{Bool}$ ,  $B(\text{false}) = \text{Nat}$ ,  $B(\text{true}) = \text{Float}$ , то

$$\Pi x : \text{Bool}. B(x) \cong \text{Nat} \times \text{Float}.$$

- Как обобщение пространства функций. Обычный тип функций  $A \rightarrow B$  — частный случай  $\Pi x : A. B$ .
- Примеры в синтаксисе Coq:
  - $\text{le\_n} : \forall (n : \text{nat}), n \leq n$
  - $\text{sorted\_inv} : \forall (x : \mathbb{Z}) (l : \text{list } \mathbb{Z}), \text{sorted}(x :: l) \Rightarrow \text{sorted}(l)$



# Простой пример: векторы фиксированной длины

## Идея

Тип вектора зависит от его длины:  $\text{Vec } A \ n$ , где  $n : \mathbb{N}$ .

- $\text{Vec} : \star \rightarrow \mathbb{N} \rightarrow \star$  — семейство типов
- $\text{nil} : \text{Vec } A \ 0$  — пустой вектор
- $\text{cons} : A \rightarrow \text{Vec } A \ n \rightarrow \text{Vec } A \ (n + 1)$  — добавление элемента

## Типобезопасные операции

- $\text{head} : \prod n : \mathbb{N}. \text{Vec } A \ (n + 1) \rightarrow A$  — гарантированно не пустой
- $\text{append} : \prod n \ m : \mathbb{N}. \text{Vec } A \ n \rightarrow \text{Vec } A \ m \rightarrow \text{Vec } A \ (n + m)$

Компилятор отвергнет  $\text{head nil}$  на этапе проверки типов.

## Пример: типобезопасный printf

Идея: тип результата зависит от формата строки

```
(* Type family for format *)
Fixpoint printf_type (s : string) : Set :=
  match s with
  | nil      => Nat
  | cons "%d" r => Int -> printf_type r
  | cons _   r => printf_type r
  end.

(* Main function has a dependent type *)
printf : forall s : string, printf_type s.
```

# Система $\lambda P$ : правила вывода

$$\begin{array}{c} \frac{}{\vdash \star : \square} \text{ (sort)} \qquad \frac{\Gamma \vdash A : s \quad x \notin \Gamma}{\Gamma, x:A \vdash x : A} \text{ (var)} \qquad \frac{\Gamma \vdash M : B \quad \Gamma \vdash A : s \quad x \notin \Gamma}{\Gamma, x:A \vdash M : B} \text{ (weak)} \\[10pt] \frac{\Gamma \vdash A : s \quad \Gamma, x:A \vdash B : s}{\Gamma \vdash \Pi x:A. B : s} \text{ (form)} \qquad \frac{\Gamma \vdash M : \Pi x:A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]} \text{ (appl)} \\[10pt] \frac{\Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda x:A. M : \Pi x:A. B} \text{ (abst)} \qquad \frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s \quad A =_{\beta} B}{\Gamma \vdash M : B} \text{ (conv)} \end{array}$$

- $\Gamma \vdash \Pi x:A. P\ x : \star$  — тип всеобщего утверждения (часто необитаем, «похоже на»  $\perp$ ).
- $\Gamma \vdash \Pi x:A. P\ x \rightarrow P\ x : \star$  — top type (всегда обитаем).
- Обитатель:

$$\lambda x:A. \lambda y:P(x). y \ : \ \Pi x:A. P(x) \rightarrow P(x).$$

## $\lambda P$ : соответствие логике предикатов

| Минимальная логика предикатов | Теория типов $\lambda P$         |
|-------------------------------|----------------------------------|
| $S$ — множество               | $S : \star$                      |
| $A$ — утверждение             | $A : \star$                      |
| $a \in S$                     | $a : S$                          |
| $p$ доказывает $A$            | $p : A$                          |
| $P$ — предикат на $S$         | $P : S \rightarrow \star$        |
| $A \Rightarrow B$             | $A \rightarrow B (= \Pi x:A. B)$ |
| $\forall x \in S. P(x)$       | $\Pi x:S. P(x)$                  |
| $(\Rightarrow$ -elim)         | (appl)                           |
| $(\Rightarrow$ -intro)        | (abst)                           |
| $(\forall$ -elim)             | (appl)                           |
| $(\forall$ -intro)            | (abst)                           |

## $\lambda P$ : пример вывода (логическая форма)

Доказываем:  $\forall x \in S. \forall y \in S. Q(x, y) \Rightarrow \forall u \in S. Q(u, u)$

$$\frac{\frac{\frac{z : \forall x \in S. \forall y \in S. Q(x, y)}{\forall y \in S. Q(u, y)} (\forall\text{-elim})}{Q(u, u)} u \in S (\forall\text{-elim})}{\forall u \in S. Q(u, u)} (\forall\text{-intro})}{\forall x \in S. \forall y \in S. Q(x, y) \Rightarrow \forall u \in S. Q(u, u)} (\Rightarrow\text{-intro})$$

Идея: предполагаем  $z$ , берём произвольный  $u$ , применяем  $z$  к  $u$  дважды (( $\forall$ -elim)), абстрагируемся (( $\forall$ -intro), ( $\Rightarrow$ -intro)).

## $\lambda$ P: пример вывода (типовая форма)

Выводим:  $\lambda z. \lambda u. z u u : (\Pi x:S. \Pi y:S. Q x y) \rightarrow \Pi u:S. Q u u$

Дано:  $S : \star, Q : S \rightarrow S \rightarrow \star$

$$\frac{\frac{\frac{z : \Pi x:S. \Pi y:S. Q x y \in \Gamma}{\Gamma \vdash z : \Pi x:S. \Pi y:S. Q x y} \text{ (var)}}{\Gamma \vdash z u : \Pi y:S. Q u y} \text{ (appl)} \quad \frac{u : S \in \Gamma}{\Gamma \vdash u : S} \text{ (var)} \quad \Gamma \vdash u : S}{\Gamma, z, u \vdash z u u : Q u u} \text{ (appl)} \quad \Gamma, z, u \vdash z u u : Q u u}{\Gamma, z \vdash \lambda u:S. z u u : \Pi u:S. Q u u} \text{ (abst)} \quad \Gamma, z \vdash \lambda u:S. z u u : \Pi u:S. Q u u}{\Gamma \vdash \lambda z. \lambda u. z u u : (\Pi x:S. \Pi y:S. Q x y) \rightarrow \Pi u:S. Q u u} \text{ (abst)}$$

Идея: применяем  $z$  к  $u$  дважды (два раза (appl)), затем абстрагируемся по  $u$  и  $z$  ((abst)).

# План лекции

- 1 Система  $\lambda\omega$
- 2 Зависимые типы и система  $\lambda P$
- 3 Зависимые суммы, уточнения и индуктивные типы



## $\Sigma$ -тип: интуиция

### Обычная пара

$A \times B$  — пара, где первый элемент из  $A$ , второй из  $B$ .

Типы компонент независимы.

### Зависимая пара

$\Sigma x:A. B(x)$  — пара, где первый элемент  $a : A$ , а второй элемент имеет тип  $B(a)$ .

Тип второго компонента зависит от значения первого.

### Пример

$\Sigma n : \mathbb{N}. \text{Vec Int } n$  — пара из числа  $n$  и вектора длины именно  $n$ .

Если первый компонент  $= 5$ , то второй имеет тип  $\text{Vec Int } 5$ .

# $\Sigma$ -тип: определение

## Определение ( $\Sigma$ -тип)

Зависимая пара (dependent pair type, dependent sum type) — тип вида

$$\Sigma_{x:A} B(x) \equiv \Sigma_{x:A}. B(x),$$

где  $A : \star$  — тип, а  $B : A \rightarrow \star$  — семейство типов, индексированное элементами  $A$ .

- Если  $B : A \rightarrow \star$  — константная функция (т.е.  $B$  не зависит от  $x$ ), то

$$\Sigma_{x:A} B \simeq A \times B.$$

- $\Sigma$ -тип можно рассматривать как размеченное объединение (disjoint union).
- Если  $A = \text{Bool}$ ,  $B(\text{false}) = \text{Nat}$ ,  $B(\text{true}) = \text{Float}$ , то

$$\Sigma_{b:\text{Bool}} B(b) \cong \text{Nat} + \text{Float}.$$

## $\Sigma$ -тип: правила вывода

$$\frac{\Gamma \vdash A : \star \quad \Gamma, x:A \vdash B(x) : \star}{\Gamma \vdash \Sigma_{x:A} B(x) : \star} \text{ (\textcolor{blue}{\Sigma-form})}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma_{x:A} B(x)} \text{ (\textcolor{blue}{\Sigma-intro})}$$

$$\frac{\Gamma \vdash z : \Sigma_{x:A} B(x)}{\Gamma \vdash \pi_1(z) : A} \text{ (\textcolor{red}{\Sigma-elim}_1)}$$

$$\frac{\Gamma \vdash z : \Sigma_{x:A} B(x)}{\Gamma \vdash \pi_2(z) : B(\pi_1(z))} \text{ (\textcolor{red}{\Sigma-elim}_2)}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \pi_1(a, b) \equiv a : A} \text{ (comp}_1\text{)}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \pi_2(a, b) \equiv b : B(\pi_1(a, b))} \text{ (comp}_2\text{)}$$

Порядок:  $\text{formation/introduction}$ ,  $\text{elimination}$ , вычислительные правила

## $\Sigma$ -тип: конкретный пример

### Задача

Функция, которая возвращает элемент списка и доказательство, что он в списке.

- Тип результата:

$$\Sigma x : A. (x \in \text{list})$$

- Первый компонент: элемент  $x : A$
- Второй компонент: доказательство  $p : x \in \text{list}$

### Сравнение с обычной парой

- Обычная пара  $A \times \text{Proof}$  — второй компонент может быть доказательством чего угодно
- Зависимая пара  $\Sigma x : A. (x \in \text{list})$  — второй компонент обязан быть доказательством для конкретного  $x$

## $\Sigma$ и $\Pi$ типы: пример

### Утверждение в логике предикатов

$$\forall(n : \text{nat}). \forall(m : \text{nat}). (n < m \Rightarrow \exists(k : \text{nat}). n + k = m)$$

### Соответствующий тип в $\lambda P$

$$\Pi n : \text{nat}. \Pi m : \text{nat}. ((n < m) \rightarrow \Sigma k : \text{nat}. (n + k = m))$$

- $\forall$  переводится в  $\Pi$
- $\exists$  переводится в  $\Sigma$
- Импликация  $\Rightarrow$  переводится в тип функции  $\rightarrow$

# Уточнённые (refinement) типы

## Определение (Уточнённый тип)

Уточнённый тип (refinement type) — тип вида  $\{x : A \mid P(x)\}$ , где  $A$  — базовый тип, а  $P : A \rightarrow \star$  — предикат, который должен выполняться для любого элемента уточнённого типа.

- Уточнённые типы выражают предусловия и постусловия.
- Связь с  $\Sigma$ -типом:  $\{x : A \mid P(x)\} \equiv \Sigma x:A. P(x)$ .

## Пример: функция, сокращающая вектор

```
-- ('a -> bool) -> {v : Vec 'a n} -> {v' : Vec 'a m | m < n}
```

# Индукция не выводима в $\lambda C$

## Индуктивное определение натуральных чисел

$$\text{nat} ::= 0 \mid \text{succ } n$$

- Принцип математической индукции можно выразить как тип:

$$\text{ind} := \Pi P : \text{nat} \rightarrow \star. (P\ 0 \rightarrow (\Pi y : \text{nat}. P\ y \rightarrow P(\text{succ } y)) \rightarrow \Pi x : \text{nat}. P\ x)$$

## Ограничение $\lambda C$

Тип  $\text{ind}$  необитаем в системе  $\lambda C$ . Более того, любой нетривиальный индуктивный тип необитаем в  $\lambda C$ .

См.: Geuvers H. (2001). Induction Is Not Derivable in Second Order Dependent Type Theory. TLCA 2001.

Интуиция: в  $\lambda C$  нет примитивных индуктивных принципов. Общая индукция требует индуктивных типов/элиминаторов сверх  $\lambda C$ , поэтому  $\text{ind}$  там необитаем.

## Определение (W-тип)

W-тип (well-founded tree type) — один из способов задания индуктивных типов. Для  $A : \star$  и  $B : A \rightarrow \star$  определяется тип

$$W_{x:A} B(x),$$

элементы которого — деревья, где каждый узел помечен  $a : A$  и имеет  $B(a)$  поддеревьев.

- $A$  — множество меток конструкторов
- $B : A \rightarrow \star$  — арность каждого конструктора
- W-типы обобщают натуральные числа, списки и древовидные структуры

## Примеры

- $\mathbb{N} \cong W(2, f)$ , где  $f(\text{inl}(\star)) = 0$ ,  $f(\text{inr}(\star)) = 1$
- $\text{List}(A) \cong W(1 + A, f)$ , где  $f(\text{inl}(\star)) = 0$ ,  $f(\text{inr}(a)) = 1$



# Пример индуктивного типа: бинарные деревья

## Формирование типа и конструкторы

- $\text{tree} : \star$  — тип бинарных деревьев
- $\text{leaf} : \text{tree}$  — конструктор листа
- $\text{node} : \text{tree} \rightarrow \text{tree} \rightarrow \text{tree}$  — конструктор узла

## Элиминатор (принцип рекурсии)

Для произвольного семейства  $P : \text{tree} \rightarrow \star$ :

$$\text{rec}_{\text{tree}} : (P \text{ leaf}) \rightarrow (\Pi t_1 : \text{tree}. \Pi t_2 : \text{tree}. P t_1 \rightarrow P t_2 \rightarrow P (\text{node } t_1 t_2)) \rightarrow \Pi t : \text{tree}. P t$$

Для доказательства  $P(t)$  для любого дерева  $t$  достаточно рассмотреть базовый случай (лист) и индуктивный шаг (узел).

# Конкретный пример: натуральные числа как W-тип

## Кодирование $\mathbb{N}$

Натуральные числа можно представить как W-тип:

$$\mathbb{N} \cong W_{x:2} B(x)$$

где  $2 = \{\text{zero}, \text{succ}\}$  и

$$B(x) = \begin{cases} 0 & \text{если } x = \text{zero} \\ 1 & \text{если } x = \text{succ} \end{cases}$$

Интуиция:

- `zero` — конструктор без аргументов ( $B(\text{zero}) = 0$ )
- `succ` — конструктор с одним аргументом ( $B(\text{succ}) = 1$ )
- Число 3: `succ(succ(succ(zero)))`

Конструктор:

- `sup(zero, f)` где  $f : 0 \rightarrow \mathbb{N}$  кодирует 0
- `sup(succ, f)` где  $f : 1 \rightarrow \mathbb{N}$  кодирует  $S(f(\star))$

# Конкретный пример: списки как W-тип

## Кодирование List(A)

Списки над A можно представить как W-тип:

$$\text{List}(A) \cong W_{x:1+A} B(x)$$

где

$$B(x) = \begin{cases} 0 & \text{если } x = \text{inl}(\star) \quad (\text{nil}) \\ 1 & \text{если } x = \text{inr}(a) \quad (\text{cons}) \end{cases}$$

Интуиция:

- `nil` — конструктор без аргументов
- `cons(a, xs)` — конструктор с элементом и рекурсией

Пример: список [1, 2, 3]

$$\text{sup}(\text{inr}(1), \lambda \star . \text{sup}(\text{inr}(2), \dots))$$

# Конкретный пример: бинарные деревья как W-тип

## Кодирование $\text{BTree}(A)$

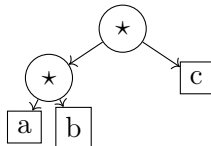
Бинарные деревья с метками  $A$  в листьях:

$$\text{BTree}(A) \cong W_{x:A+1} B(x)$$

где

$$B(x) = \begin{cases} 0 & x = \text{inl}(a) \quad (\text{лист}) \\ 2 & x = \text{inr}(\star) \quad (\text{узел}) \end{cases}$$

Визуализация:



Узлы  $(\circ)$  — арность 2, листья  $(\square)$  — арность 0

# Элиминаторы W-типов: универсальное свойство

## Принцип рекурсии для W-типов

Для  $W_{x:A} B(x)$  и семейства  $P : W_{x:A} B(x) \rightarrow \star$  определён элиминатор:

$$\begin{aligned} \text{rec}_W : & (\Pi a : A. \Pi f : (B(a) \rightarrow W). \\ & (\Pi b : B(a). P(f(b)))) \rightarrow P(\text{sup}(a, f)) \\ & \rightarrow \Pi w : W. P(w) \end{aligned}$$

Интуиция:

- 1 Берём конструктор  $a : A$  и его аргументы  $f : B(a) \rightarrow W$
- 2 Рекурсивно вычисляем  $P$  для всех поддеревьев
- 3 Комбинируем результаты для  $a$

## Связь с индукцией

Элиминатор кодирует структурную индукцию.

## Связь элиминаторов и свёрток

Элиминатор W-типа — это обобщение катаморфизма (свёртки).

Напомним из лекции 5: катаморфизм — это структурная рекурсия, которая «сворачивает» структуру данных в результат.

Свёртка для списков:

$$\text{fold} : (A \rightarrow X \rightarrow X) \rightarrow X \rightarrow \text{List}(A) \rightarrow X$$

Пример:

$$\text{fold}(c, n, [a_1, \dots, a_n])$$

Элиминатор для списков:

$$\text{rec}_{\text{List}} : \dots \rightarrow \prod_{xs} P(xs)$$

При  $P = \lambda\_ . X$ :

$$\text{rec}_{\text{List}} = \text{fold}$$

**Вывод:** элиминаторы W-типов обобщают свёртки на зависимые типы.

# Катаморфизмы: примеры на натуральных числах

## Свёртка натуральных чисел

Для  $\mathbb{N}$  катаморфизм задаётся парой  $(z : X, s : X \rightarrow X)$ :

$$\text{fold}_{\mathbb{N}}(z, s) : \mathbb{N} \rightarrow X$$

Вычислительные правила:

$$\begin{aligned}\text{fold}_{\mathbb{N}}(z, s)(0) &= z \\ \text{fold}_{\mathbb{N}}(z, s)(S(n)) &= s(\text{fold}_{\mathbb{N}}(z, s)(n))\end{aligned}$$

Конкретные примеры:

- $\text{add}(n, m) = \text{fold}_{\mathbb{N}}(m, S)(n)$  — сложение
- $\text{mul}(n, m) = \text{fold}_{\mathbb{N}}(0, \lambda x. \text{add}(m, x))(n)$  — умножение
- $\text{exp}(n, m) = \text{fold}_{\mathbb{N}}(1, \lambda x. \text{mul}(n, x))(m)$  — возведение в степень
- $\text{isEven} : \mathbb{N} \rightarrow \text{Bool}$  через  $\text{fold}_{\mathbb{N}}(\text{true}, \text{not})$

# Катаморфизмы на списках

## Свёртка списка (fold)

Катаморфизм задаётся алгеброй  $\alpha : 1 + A \times X \rightarrow X$ :

$$\text{fold}_{\text{List}}(n : X, c : A \rightarrow X \rightarrow X) : \text{List}(A) \rightarrow X$$

Вычислительные правила:

$$\begin{aligned}\text{fold}_{\text{List}}(n, c)(\text{nil}) &= n \\ \text{fold}_{\text{List}}(n, c)(\text{cons}(a, xs)) &= c(a, \text{fold}_{\text{List}}(n, c)(xs))\end{aligned}$$

Примеры в Haskell:

```
sum    = foldr (+) 0
length = foldr (\_ n -> n+1) 0
map f  = foldr (\x xs -> f x : xs) []
```

sum — сумма, length — длина, map — отображение



# От катаморфизмов к элиминаторам

## Структурная рекурсия

Для индуктивного типа  $T$  катаморфизм — это функция  $h : T \rightarrow X$ , определённая структурной рекурсией:

- Для каждого конструктора с задаётся правило вычисления
- Рекурсивно применяется к подструктурам
- Результаты комбинируются в ответ типа  $X$

## Элиминатор зависимых типов

Элиминатор обобщает катаморфизм:

- Катаморфизм:  $T \rightarrow X$  для фиксированного  $X$
- Элиминатор:  $\Pi t : T. P(t)$  для семейства  $P : T \rightarrow \star$

При  $P = \lambda \_ . X$  элиминатор совпадает с катаморфизмом.

**Вывод:** катаморфизмы — вычисления, элиминаторы — доказательства.

# Структурная рекурсия и завершимость

## Почему $W$ -типы завершаются

Все функции, определённые через элиминаторы  $W$ -типов, тотальны (завершаются).

Причина: структурная рекурсия всегда уменьшает «размер» аргумента.

Пример: сложение натуральных чисел

$$\text{add}(n, m) = \text{fold}_{\mathbb{N}}(m, S)(n)$$

- На каждом шаге рекурсии  $n$  уменьшается
- Базовый случай  $0$  достигается за конечное число шагов
- Функция гарантированно завершается

## Связь с теоремой Жирара

В ЛС (и системах с  $W$ -типами) невозможно определить расходящиеся вычисления — отсюда сильная нормализация.

# Классификация по парам сортов

## Идея

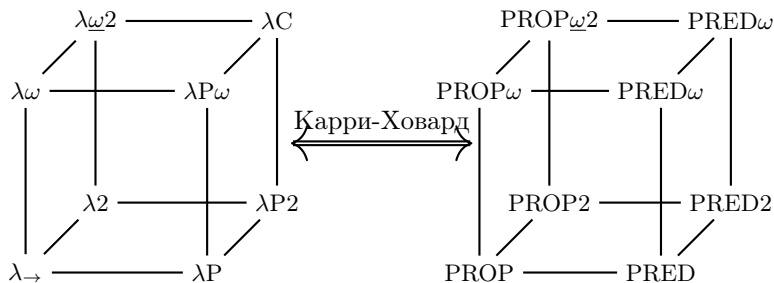
Каждая система типов определяется набором допустимых пар сортов  $(s_1, s_2)$ , где  $s_i \in \{\star, \square\}$ .

Если  $x : A : s_1$  и  $b : B : s_2$ , то  $\lambda x : A. b$  имеет тип, зависящий от  $s_1$  и  $s_2$ .

| $x : A : s_1$ | $b : B : s_2$ | $(s_1, s_2)$         | Зависимость     | Система                 |
|---------------|---------------|----------------------|-----------------|-------------------------|
| $\star$       | $\star$       | $(\star, \star)$     | термы от термов | $\lambda_{\rightarrow}$ |
| $\square$     | $\star$       | $(\square, \star)$   | термы от типов  | $\lambda_2$             |
| $\square$     | $\square$     | $(\square, \square)$ | типы от типов   | $\lambda_{\omega}$      |
| $\star$       | $\square$     | $(\star, \square)$   | типы от термов  | $\lambda_P$             |

Замечание: ЛС (Calculus of Constructions) включает все четыре пары:  $\{(\star, \star), (\square, \star), (\square, \square), (\star, \square)\}$

# Куб Барендрегта: соответствие с логиками



Системы типов (слева):

- $\lambda_{\rightarrow}$  — просто типизированное  $\lambda$ -исчисление
- $\lambda 2$  — System F (полиморфизм)
- $\lambda_{\omega}$  — конструкторы типов
- $\lambda P$  — зависимые типы
- $\lambda C$  — Calculus of Constructions

Логики (справа):

- $PROP$  — пропозициональная логика
- $PROP 2$  — логика 2-го порядка
- $PRED$  — логика предикатов
- $PRED_{\omega}$  — логика высших порядков